

MockK

mocking library for Kotlin

[View on GitHub](#)

[Try effective-retro.com - free retrospective boards →](#)



Kotlin

release [v1.14.9](#)

[change log](#)

[codecov](#) [75%](#)

[android](#) [support](#)

[matrix](#) [test](#)

[code helpers](#) [34](#)

Getting started

All you need to get started is just to add a dependency to [MockK](#) library.

Gradle/Maven dependency

Approach	Instruction
	<pre>testImplementation "io.mockk:mockk:\${mockkVersion}"</pre>

Approach	Instruction
 (Kotlin DSL)	<pre>testImplementation("io.mockk:mockk:\${mockkVersion}")</pre>
	<pre><dependency> <groupId>io.mockk</groupId> <artifactId>mockk-jvm</artifactId> <version>\${mockkVersion}</version> <scope>test</scope> </dependency></pre>
	<pre>testImplementation "io.mockk:mockk-android:\${mockkVersion}" testImplementation "io.mockk:mockk-agent:\${mockkVersion}"</pre>
	<pre>androidTestImplementation "io.mockk:mockk-android:\${mockkVersion}" androidTestImplementation "io.mockk:mockk-agent:\${mockkVersion}"</pre>

DSL examples

Simplest example. By default mocks are strict, so you need to provide some behaviour.

```
val car = mockk<Car>()

every { car.drive(Direction.NORTH) } returns Outcome.OK

car.drive(Direction.NORTH) // returns OK

verify { car.drive(Direction.NORTH) }

confirmVerified(car)
```

See the “Features” section below for more detailed examples.

BDD style (optional)

For teams using Behavior-Driven Development, MockK provides BDD-style aliases

```
testImplementation "io.mockk:mockk:${mockkVersion}"  
testImplementation "io.mockk:mockk-bdd:${mockkVersion}"
```

```
androidTestImplementation "io.mockk:mockk-android:${mockkVersion}"  
androidTestImplementation "io.mockk:mockk-bdd-android:${mockkVersion}"
```

BDD aliases

Standard MockK	BDD style
<code>every { ... }</code>	<code>given { ... }</code>
<code>coEvery { ... }</code>	<code>coGiven { ... }</code>
<code>verify { ... }</code>	<code>then { ... }</code>
<code>coVerify { ... }</code>	<code>coThen { ... }</code>

Spring support

- [springmockk](#) introduced in official [Spring Boot Kotlin tutorial](#)

Quarkus support

- [quarkus-mockk](#) adds support for mocking beans in Quarkus. Documentation can be found [here](#)

Kotlin version support

From version 1.13.0 MockK supports Kotlin 1.4 and higher

Known issues

- PowerMock needs a workaround to run together with MockK [#79](#). (not sure after workaround if it is generally usable or not, please somebody report it)
- Inline functions cannot be mocked: see the discussion on [this issue](#)

- Spies, `mockkStatic` may not work on JDK 16+;
`InaccessibleObjectException` / `IllegalAccessException`: [read more here](#)
- Using a spy with a suspending function [will give unexpected test results](#)

Table of contents:

- Getting started
 - Gradle/Maven dependency
- DSL examples
- BDD style (optional)
 - BDD aliases
 - Spring support
 - Quarkus support
 - Kotlin version support
 - Known issues
- Examples, guides & articles
 - Kotlin Academy articles 
 - Japanese guides and articles
 - Chinese guides and articles
 - Korean guides and articles
- Features
 - Annotations
 - `@InjectMockKs` dependency order
 - JUnit4
 - JUnit5
 - Automatic verification confirmation
 - Automatic unnecessary stubbing check
 - Declarative enabling of dependency order
 - Spy
 - Relaxed mock
 - Partial mocking
 - Mock relaxed for functions returning Unit
 - Object mocks
 - Class mock
 - Enumeration mocks
 - Constructor mocks
 - Partial argument matching
 - Chained calls
 - Hierarchical mocking
 - Capturing
 - Verification `atLeast`, `atMost` or exactly times

- Verification order
- Verification confirmation
- Unnecessary stubbing
- Recording exclusions
- Verification timeout
- Returning Unit
- Coroutines
- Top Level functions
- Extension functions
- Varargs
- Private functions mocking / dynamic calls
- Property backing fields
- Multiple interfaces
- Mocking Nothing
- Clearing vs Unmocking
- Scoped mocks
- Suppressing superclass calls
- Matcher extensibility
 - Custom matchers
 - Reflection matchers
- Configuration File
 - How to use:
 - Configuration Options
 - Legacy Configuration
- DSL tables
 - Top level functions
 - Matchers
 - Validators
 - Answers
 - Additional answer(s)
 - Answer scope
 - Vararg scope
- Restricted Mocking in MockK
 - Overview
 - Why Restrict Mocking?
 - Default Restricted Classes
 - How to Configure Restricted Mocking
 - Configure Restricted Mocking with System Properties
 - Configure Restricted Mocking with Gradle Properties
 - Behavior When Mocking Restricted Classes

- When `mockk.throwExceptionOnBadMock=false` (Default)
- When `mockk.throwExceptionOnBadMock=true`
- Custom Class Restriction Example
 - Example 1: Mocking a Restricted Custom Class (Throws Exception)
- Funding
 - Sponsors
 - Backers
 - Contributors
- Getting Help

Examples, guides & articles

Kotlin Academy articles

Check the series of articles “Mocking is not rocket science” at [Kt. Academy](#) describing MockK from the very basics of mocking up to description of all advanced features.

- Basics
- Expected behavior and behavior verification
- MockK features
- MockK advanced features
- Testing Quarkus with Kotlin, JUnit and MockK
- Unraveling MockK’s black magic(EN, translation)
- Mockk Guidebook
- “Kotlin Unit Testing with Mockk” by Marco Cattaneo
- (Video) Use verify in MockK to validate function calls on mocked object
- Testing With MockK paid course on [raywenderlich.com](#)
- TDD for Android video tutorial [part 1](#), [part 2](#) by Ryan Kay
- (Video)Android Developer Live Coding #13: Unit Testing with Mockk, Coroutines, Test Driven Development
- KotlinConf 2018 - Best Practices for Unit Testing in Kotlin by Philipp Hauer
- [kotlin-fullstack-sample](#) uses MockK project covered with tests
- DZone article
- Habrahabr article (RU)
- Mocking in Kotlin with MockK - Yannick De Turck
- How does MockK work internally? - Sagar Malhotra

Japanese guides and articles

- Documentation translation to Japanese
- Unraveling MockK's black magic / MockKの「[モック](#)」を[理解](#)する (JP, but readable through chrome translator)

Chinese guides and articles

- [Kotlin + Mockito](#)
- [MockK mockk, every, Annotation, verify](#)
- [MockK Relaxed Mocks, Verify, Capture](#)
- [MockK Static Method, Singleton](#)

Korean guides and articles

- Documentation translation to Korean
- [MockK의 흑마술을 파헤치자!](#)

Features

Annotations

You can use annotations to simplify the creation of mock objects:

```
class TrafficSystem {
    lateinit var car1: Car

    lateinit var car2: Car

    lateinit var car3: Car
}

class CarTest {
    @MockK
    lateinit var car1: Car

    @RelaxedMockK
    lateinit var car2: Car

    @MockK(relaxUnitFun = true)
    lateinit var car3: Car
}
```

```

@SpyK
var car4 = Car()

@InjectMockKs
var trafficSystem = TrafficSystem()

@Before
fun setUp() = MockKAnnotations.init(this, relaxUnitFun = true) // turn relaxUnitFu

@Test
fun calculateAddsValues1() {
    // ... use car1, car2, car3 and car4
}
}

```

Injection first tries to match properties by name, then by class or superclass. Check the `lookupType` parameter for customization.

Properties are injected even if `private` is applied. Constructors for injection are selected from the biggest number of arguments to lowest.

`@InjectMockKs` by default injects only `lateinit var`s or `var`s that are not assigned. To change this, use `overrideValues = true`. This would assign the value even if it is already initialized somehow. To inject `val`s, use `injectImmutable = true`. For a shorter notation use `@OverrideMockKs` which does the same as `@InjectMockKs` by default, but turns these two flags on.

@InjectMockKs dependency order

If multiple `@InjectMockKs` properties depend on each other via constructor parameters, initialization order matters. By default, MockK processes them in reflection order; to resolve dependencies deterministically, enable dependency order:

```

@Before
fun setUp() = MockKAnnotations.init(this, useDependencyOrder = true)

```

This applies a topological sort across `@InjectMockKs` and throws `MockKException` on circular dependencies. Enabling `useDependencyOrder` adds an approximate 3-5% performance overhead during initialization.

JUnit4

JUnit 4 exposes a rule-based API to allow for some automation following the test lifecycle. MockK includes a rule which uses this to set up and tear down your mocks without needing to manually call `MockKAnnotations.init(this)`. Example:

```
class CarTest {
    @get:Rule
    val mockkRule = MockKRule(this)

    @MockK
    lateinit var car1: Car

    @RelaxedMockK
    lateinit var car2: Car

    @Test
    fun something() {
        every { car1.drive() } just runs
        every { car2.changeGear(any()) } returns true
        // etc
    }
}
```

JUnit5

In JUnit5 you can use `MockKExtension` to initialize your mocks.

```
@ExtendWith(MockKExtension::class)
class CarTest {
    @MockK
    lateinit var car1: Car

    @RelaxedMockK
    lateinit var car2: Car

    @MockK(relaxUnitFun = true)
    lateinit var car3: Car

    @SpyK
    var car4 = Car()

    @Test
    fun calculateAddsValues1() {
        // ... use car1, car2, car3 and car4
    }
}
```

```
}  
}
```

Additionally, it adds the possibility to use `@MockK` and `@RelaxedMockK` on test function parameters:

```
@Test  
fun calculateAddsValues1(@MockK car1: Car, @RelaxedMockK car2: Car) {  
    // ... use car1 and car2  
}
```

Finally, this extension will call `unmockkAll` and `clearAllMocks` in a `@AfterAll` callback, ensuring your test environment is clean after each test class execution. You can disable this behavior by adding the `@MockKExtension.KeepMocks` annotation to your class or globally by setting the `mockk.junit.extension.keepmocks=true` property. (Since v1.13.11) Alternatively, since `clearAllMocks` by default (`currentThreadOnly=false`) is not thread-safe, if you need to run test in parallel you can add the `MockKExtension.RequireParallelTesting` annotation to your class or set the `mockk.junit.extension.requireParallelTesting=true` property to disable calling it in the `@AfterAll` callback. If `clearAllMocks` is explicitly called, you can supply `clearAllMocks(currentThreadOnly = true)` so that it only clears mocks created within the same thread (since v1.13.12).

Automatic verification confirmation

You can make sure that all stubbed methods are actually verified by also annotating your test class with `@MockKExtension.ConfirmVerification`.

This will internally call `confirmVerified` on all mocks after each test, to make sure there are no unnecessary stubbings.

Please note that this behavior may not work as expected when running tests in your IDE, as it is Gradle who takes care of handling the exception being thrown when these `confirmVerified` calls fail.

Automatic unnecessary stubbing check

You can make sure that all stubbed methods are useful - used at least once - by also annotating your test class with `@MockKExtension.CheckUnnecessaryStub`.

This will internally call `checkUnnecessaryStub` on all mocks after each test, to make sure there are no unnecessary stubbings.

Declarative enabling of dependency order

You can enable topological dependency resolution of `@InjectMockKs` by also annotating your test class with `@MockKExtension.UseDependencyOrder`.

This will internally process each test instance of the annotated class with `MockKAnnotations.init(useDependencyOrder==true)`.

Spy

Spies allow you to mix mocks and real objects.

```
val car = spyk(Car()) // or spyk<Car>() to call the default constructor

car.drive(Direction.NORTH) // returns whatever the real function of Car returns

verify { car.drive(Direction.NORTH) }

confirmVerified(car)
```

Note 1: the spy object is a copy of the passed object. Note 2: there is a known issue if using a spy with a suspending function: <https://github.com/mockk/mockk/issues/554>

Relaxed mock

A `relaxed mock` is the mock that returns some simple value for all functions. This allows you to skip specifying behavior for each case, while still stubbing things you need. For reference types, chained mocks are returned.

```
val car = mockk<Car>(relaxed = true)

car.drive(Direction.NORTH) // returns null

verify { car.drive(Direction.NORTH) }

confirmVerified(car)
```

Note: relaxed mocking is working badly with generic return types. A class cast exception is usually thrown in this case. Opt for stubbing manually in the case of a generic return type.

Workaround:

```

val func = mockk<() -> Car>(relaxed = true) // in this case invoke function has gene

// this line is workaround, without it the relaxed mock would throw a class cast exc
every { func() } returns Car() // or you can return mockk() for example

func()

```

Partial mocking

Sometimes, you need to stub some functions, but still call the real method on others, or on specific arguments. This is possible by passing `callOriginal()` to `answers`, which works for both relaxed and non-relaxed mocks.

```

class Adder {
    fun addOne(num: Int) = num + 1
}

val adder = mockk<Adder>()

every { adder.addOne(any()) } returns -1
every { adder.addOne(3) } answers { callOriginal() }

assertEquals(-1, adder.addOne(2))
assertEquals(4, adder.addOne(3)) // original function is called

```

Mock relaxed for functions returning Unit

If you want `Unit`-returning functions to be relaxed, you can use `relaxUnitFun = true` as an argument to the `mockk` function, `@MockK` annotation or `MockKAnnotations.init` function.

Function:

```

mockk<ClassBeingMocked>(relaxUnitFun = true)

```

Annotation:

```

@MockK(relaxUnitFun = true)
lateinit var mock1: ClassBeingMocked
init {

```

```
MockKAnnotations.init(this)
}
```

MockKAnnotations.init:

```
@MockK
lateinit var mock2: ClassBeingMocked
init {
    MockKAnnotations.init(this, relaxUnitFun = true)
}
```

Object mocks

Objects can be turned into mocks in the following way:

```
object ObjBeingMocked {
    fun add(a: Int, b: Int) = a + b
}

mockkObject(ObjBeingMocked) // applies mocking to an Object

assertEquals(3, ObjBeingMocked.add(1, 2))

every { ObjBeingMocked.add(1, 2) } returns 55

assertEquals(55, ObjBeingMocked.add(1, 2))
```

To revert back, use `unmockkObject` or `unmockkAll` (more destructive: cancels object, static and constructor mocks)

```
@Before
fun beforeTests() {
    mockkObject(ObjBeingMocked)
    every { ObjBeingMocked.add(1,2) } returns 55
}

@Test
fun willUseMockBehaviour() {
    assertEquals(55, ObjBeingMocked.add(1,2))
}

@After
```

```
fun afterTests() {
    unmockkObject(ObjBeingMocked)
    // or unmockkAll()
}
```

Despite the Kotlin language restrictions, you can create new instances of objects if required by testing logic:

```
val newObjectMock = mockk<ObjBeingMocked>()
```

Class mock

Sometimes you need a mock of an arbitrary class. Use `mockkClass` in those cases.

```
val car = mockkClass(Car::class)

every { car.drive(Direction.NORTH) } returns Outcome.OK

car.drive(Direction.NORTH) // returns OK

verify { car.drive(Direction.NORTH) }
```

Enumeration mocks

Enums can be mocked using `mockkObject`:

```
enum class Enumeration(val goodInt: Int) {
    CONSTANT(35),
    OTHER_CONSTANT(45);
}

mockkObject(Enumeration.CONSTANT)
every { Enumeration.CONSTANT.goodInt } returns 42
assertEquals(42, Enumeration.CONSTANT.goodInt)
```

Constructor mocks

Sometimes, especially in code you don't own, you need to mock newly created objects. For this purpose, the following constructs are provided:

```

class MockCls {
    fun add(a: Int, b: Int) = a + b
}

mockkConstructor(MockCls::class)

every { anyConstructed<MockCls>().add(1, 2) } returns 4

assertEquals(4, MockCls().add(1, 2)) // note new object is created

verify { anyConstructed<MockCls>().add(1, 2) }

```

The basic idea is that just after the constructor of the mocked class is executed (any of them), objects become a `constructed mock`. Mocking behavior of such a mock is connected to the special `prototype mock` denoted by `anyConstructed<MockCls>()`. There is one instance per class of such a `prototype mock`. Call recording also happens to the `prototype mock`. If no behavior for the function is specified, then the original function is executed.

In case a class has more than one constructor, each can be mocked separately:

```

class MockCls(private val a: Int = 0) {
    constructor(x: String) : this(x.toInt())
    fun add(b: Int) = a + b
}

mockkConstructor(MockCls::class)

every { constructedWith<MockCls>().add(1) } returns 2
every {
    constructedWith<MockCls>(OfTypeMatcher<String>(String::class)).add(2) // Mocks t
} returns 3
every {
    constructedWith<MockCls>(EqMatcher(4)).add(any()) // Mocks the constructor which
} returns 4

assertEquals(2, MockCls().add(1))
assertEquals(3, MockCls("2").add(2))
assertEquals(4, MockCls(4).add(7))

verify {
    constructedWith<MockCls>().add(1)
    constructedWith<MockCls>(OfTypeMatcher<String>(String::class)).add(2)
    constructedWith<MockCls>(EqMatcher(4)).add(7)
}

```

Note that in this case, a `prototype mock` is created for every set of argument matchers passed to `constructedWith`. This means that when verifying invocations on a `prototype mock` you have to ensure that the argument matchers used are the same that where used when stubbing, i.e. the matchers in the `verify` block and in the `every` block must be the same.

When constructor arguments are mocks, prefer `SameInstanceMatcher` (reference match) and reuse the same matcher instances in both `every` and `verify`:

```
mockkConstructor(Intent::class)

val uri = mockk<Uri>()
val uriMatcher = SameInstanceMatcher(uri)

every {
    constructedWith<Intent>(EqMatcher(Intent.ACTION_VIEW), uriMatcher).setFlags(any())
} returns mockk()

verify {
    constructedWith<Intent>(EqMatcher(Intent.ACTION_VIEW), uriMatcher).setFlags(any())
}
```

Partial argument matching

You can mix both regular arguments and matchers:

```
val car = mockk<Car>()

every {
    car.recordTelemetry(
        speed = more(50),
        direction = Direction.NORTH, // here eq() is used
        lat = any(),
        long = any()
    )
} returns Outcome.RECORDED

car.recordTelemetry(60, Direction.NORTH, 51.1377382, 17.0257142)

verify { car.recordTelemetry(60, Direction.NORTH, 51.1377382, 17.0257142) }

confirmVerified(car)
```


Chained calls

You can stub chains of calls:

```
val car = mockk<Car>()

every { car.door(DoorType.FRONT_LEFT).windowState() } returns WindowState.UP

car.door(DoorType.FRONT_LEFT) // returns chained mock for Door
car.door(DoorType.FRONT_LEFT).windowState() // returns WindowState.UP

verify { car.door(DoorType.FRONT_LEFT).windowState() }

confirmVerified(car)
```

Note: if the function's return type is generic then the information about the actual type is gone. To make chained calls work, additional information is required. Most of the time the framework will catch the cast exception and do `autohinting`. In the case it is explicitly required, use `hint` before making the next call.

```
every { obj.op2(1, 2).hint(Int::class).op1(3, 4) } returns 5
```

Hierarchical mocking

From version 1.9.1 mocks may be chained into hierarchies:

```
interface AddressBook {
    val contacts: List<Contact>
}

interface Contact {
    val name: String
    val telephone: String
    val address: Address
}

interface Address {
    val city: String
    val zip: String
}

val addressBook = mockk<AddressBook> {
```

```

every { contacts } returns listOf(
    mockk {
        every { name } returns "John"
        every { telephone } returns "123-456-789"
        every { address.city } returns "New-York"
        every { address.zip } returns "123-45"
    },
    mockk {
        every { name } returns "Alex"
        every { telephone } returns "789-456-123"
        every { address } returns mockk {
            every { city } returns "Wroclaw"
            every { zip } returns "543-21"
        }
    }
)
}

```

Capturing

You can capture an argument to a `CapturingSlot` or `MutableList`.

`CapturingSlot` is usually created via factory method `slot<T : Any?>()` and is possible to capture nullable and non nullable types. `MutableList` is intended for capturing multiple values during testing.

```

enum class Direction { NORTH, SOUTH }
enum class RecordingOutcome { RECORDED }
enum class RoadType { HIGHWAY }
class Car {
    fun recordTelemetry(speed: Double, direction: Direction, roadType: RoadType?): R
        TODO("not implement for showcase")
}

val car = mockk<Car>()
// allow to capture parameter with non nullable type `Double`
val speedSlot = slot<Double>()
// allow to capture parameter with nullable type `RoadType`
val roadTypeSlot = slot<RoadType?>()
val list = mutableListOf<Double>()

every {
    car.recordTelemetry(

```

```

        speed = capture(speedSlot), // makes mock match calls with any value for `sp
        direction = Direction.NORTH, // makes mock and capturing only match calls wi
        roadType = captureNullable(roadTypeSlot), // makes mock match calls with any
    )
} answers {
    println("Speed: ${speedSlot.captured}, roadType: ${roadTypeSlot.captured}")

    RecordingOutcome.RECORDED
}

every {
    car.recordTelemetry(
        speed = capture(list),
        direction = Direction.SOUTH,
        roadType = captureNullable(roadTypeSlot),
    )
} answers {
    println("Speed: ${list}, roadType: ${roadTypeSlot.captured}")

    RecordingOutcome.RECORDED
}

car.recordTelemetry(speed = 15.0, direction = Direction.NORTH, null) // prints Speed
car.recordTelemetry(speed = 16.0, direction = Direction.SOUTH, RoadType.HIGHWAY) //

verifyOrder {
    car.recordTelemetry(speed = or(15.0, 16.0), direction = any(), roadType = null)
    car.recordTelemetry(speed = 16.0, direction = any(), roadType = RoadType.HIGHWAY)
}

confirmVerified(car)

```

Verification atLeast, atMost or exactly times

You can check the call count with the `atLeast`, `atMost` or `exactly` parameters:

```

val car = mockk<Car>(relaxed = true)

car.accelerate(fromSpeed = 10, toSpeed = 20)
car.accelerate(fromSpeed = 10, toSpeed = 30)
car.accelerate(fromSpeed = 20, toSpeed = 30)

// all pass
verify(atLeast = 3) { car.accelerate(allAny()) }

```

```

verify(atMost = 2) { car.accelerate(fromSpeed = 10, toSpeed = or(20, 30)) }
verify(exactly = 1) { car.accelerate(fromSpeed = 10, toSpeed = 20) }
verify(exactly = 0) { car.accelerate(fromSpeed = 30, toSpeed = 10) } // means no call
confirmVerified(car)

```

Or you can use `verifyCount`:

```

val car = mockk<Car>(relaxed = true)

car.accelerate(fromSpeed = 10, toSpeed = 20)
car.accelerate(fromSpeed = 10, toSpeed = 30)
car.accelerate(fromSpeed = 20, toSpeed = 30)

// all pass
verifyCount {
    (3..5) * { car.accelerate(allAny(), allAny()) } // same as verify(atLeast = 3, atMost = 5)
    1 * { car.accelerate(fromSpeed = 10, toSpeed = 20) } // same as verify(exactly = 1)
    0 * { car.accelerate(fromSpeed = 30, toSpeed = 10) } // same as verify(exactly = 0)
}

confirmVerified(car)

```

Verification order

- `verifyAll` verifies that all calls happened without checking their order.
- `verifySequence` verifies that the calls happened in a specified sequence.
- `verifyOrder` verifies that calls happened in a specific order.
- `wasNot Called` verifies that the mock (or the list of mocks) was not called at all.

```

class MockedClass {
    fun sum(a: Int, b: Int) = a + b
}

val obj = mockk<MockedClass>()
val slot = slot<Int>()

every {
    obj.sum(any(), capture(slot))
} answers {
    1 + firstArg<Int>() + slot.captured
}

```

```

obj.sum(1, 2) // returns 4
obj.sum(1, 3) // returns 5
obj.sum(2, 2) // returns 5

verifyAll {
    obj.sum(1, 3)
    obj.sum(1, 2)
    obj.sum(2, 2)
}

verifySequence {
    obj.sum(1, 2)
    obj.sum(1, 3)
    obj.sum(2, 2)
}

verifyOrder {
    obj.sum(1, 2)
    obj.sum(2, 2)
}

val obj2 = mockk<MockedClass>()
val obj3 = mockk<MockedClass>()
verify {
    listOf(obj2, obj3) wasNot Called
}

confirmVerified(obj)

```

Verification confirmation

To double-check that all calls were verified by `verify...` constructs, you can use `confirmVerified`:

```
confirmVerified(mock1, mock2)
```

Since v1.14.6 you can pass `clear = true` to also clear verification marks and recorded calls for the provided mocks after confirmation.

```
confirmVerified(mock1, mock2, clear = true)
```

It doesn't make much sense to use it for `verifySequence` and `verifyAll`, as these verification methods already exhaustively cover all calls with verification.

It will throw an exception if there are some calls left without verification.

Some calls can be excluded from this confirmation, check the next section for more details.

```
val car = mockk<Car>()

every { car.drive(Direction.NORTH) } returns Outcome.OK
every { car.drive(Direction.SOUTH) } returns Outcome.OK

car.drive(Direction.NORTH) // returns OK
car.drive(Direction.SOUTH) // returns OK

verify {
    car.drive(Direction.SOUTH)
    car.drive(Direction.NORTH)
}

confirmVerified(car) // makes sure all calls were covered with verification
confirmVerified(car, clear = true) // makes sure all calls were covered with verification
```

Unnecessary stubbing

Because clean & maintainable test code requires zero unnecessary code, you can ensure that there is no unnecessary stubs.

```
checkUnnecessaryStub(mock1, mock2)
```

It will throw an exception if there are some declared calls on the mocks that are not used by the tested code. This can happen if you have declared some really unnecessary stubs or if the tested code doesn't call an expected one.

Recording exclusions

To exclude unimportant calls from being recorded, you can use `excludeRecords`:

```
excludeRecords { mock.operation(any(), 5) }
```

All matching calls will be excluded from recording. This may be useful if you are using exhaustive verification: `verifyAll`, `verifySequence` or `confirmVerified`.

```
val car = mockk<Car>()

every { car.drive(Direction.NORTH) } returns Outcome.OK
every { car.drive(Direction.SOUTH) } returns Outcome.OK

excludeRecords { car.drive(Direction.SOUTH) }

car.drive(Direction.NORTH) // returns OK
car.drive(Direction.SOUTH) // returns OK

verify {
    car.drive(Direction.NORTH)
}

confirmVerified(car) // car.drive(Direction.SOUTH) was excluded, so confirmation is
```

Verification timeout

To verify concurrent operations, you can use `timeout = xxx`:

```
mockk<MockCls> {
    every { sum(1, 2) } returns 4

    Thread {
        Thread.sleep(2000)
        sum(1, 2)
    }.start()

    verify(timeout = 3000) { sum(1, 2) }
}
```

This will wait until one of two following states: either verification is passed or the timeout is reached.

Returning Unit

If a function returns `Unit`, you can use the `justRun` construct:

```

class MockedClass {
    fun sum(a: Int, b: Int): Unit {
        println(a + b)
    }
}

val obj = mockk<MockedClass>()

justRun { obj.sum(any(), 3) }

obj.sum(1, 1)
obj.sum(1, 2)
obj.sum(1, 3)

verify {
    obj.sum(1, 1)
    obj.sum(1, 2)
    obj.sum(1, 3)
}

```

Other ways to write `justRun { obj.sum(any(), 3) }`:

- `every { obj.sum(any(), 3) }` just Runs
- `every { obj.sum(any(), 3) }` returns Unit
- `every { obj.sum(any(), 3) }` answers { Unit }

Coroutines

To mock coroutines you need to add another dependency to the support library.

Gradle

```
testImplementation "org.jetbrains.kotlinx:kotlinx-coroutines-core:x.x"
```

Maven

```

<dependency>
  <groupId>org.jetbrains.kotlinx</groupId>
  <artifactId>kotlinx-coroutines-core</artifactId>
  <version>x.x</version>

```



```
<scope>test</scope>
</dependency>
```

Then you can use `coEvery`, `coVerify`, `coMatch`, `coAssert`, `coRun`, `coAnswers` or `coInvoke` to mock suspend functions.

```
val car = mockk<Car>()

coEvery { car.drive(Direction.NORTH) } returns Outcome.OK

car.drive(Direction.NORTH) // returns OK

coVerify { car.drive(Direction.NORTH) }
```

And to simulate a never returning `suspend` function, you can use `coJustAwait`:

```
runTest {
    val car = mockk<Car>()

    coJustAwait { car.drive(any()) } // car.drive(...) will never return

    val job = launch(UnconfinedTestDispatcher()) {
        car.drive(Direction.NORTH)
    }

    coVerify { car.drive(Direction.NORTH) }

    job.cancelAndJoin() // Don't forget to cancel the job
}
```

Note: there is a known issue if using a spy with a suspending function:
<https://github.com/mockk/mockk/issues/554>

Top Level functions

Kotlin lets you declare functions that don't belong to any class or object, called top-level functions. These calls are translated to static methods in `jvm` environments, and a special Java class is generated to hold the functions. These top-level functions can be mocked using `mockkStatic`. You just need to import the function and pass a reference as the argument:

```

import com.cars.buildCar

val testCar = Car()
mockkStatic(::buildCar)
every { buildCar() } returns testCar

assertEquals(testCar, buildCar())

verify { buildCar() }
confirmVerified(::buildCar)

```

Mocking a function will clear any existing mocks of other functions declared in the same file, equivalent to calling `clearStaticMockk` on the generated enclosing class. `confirmVerified` accepts function/property references, but scoped confirmation works only for references backed by static mocks (typically top-level or module-wide extension references used with `mockkStatic`). If you pass `clear = true` to `confirmVerified` with function/property references, only selected references are cleared. `confirmVerified()` with no arguments keeps its original behavior and confirms all stubs.

Extension functions

There are three types of extension function in Kotlin:

- class-wide
- object-wide
- module-wide

For an object or a class, you can mock extension functions just by creating a regular `mockk`:

```

data class Obj(val value: Int)

class Ext {
    fun Obj.extensionFunc() = value + 5
}

with(mockk<Ext>()) {
    every {
        Obj(5).extensionFunc()
    } returns 11

    assertEquals(11, Obj(5).extensionFunc())

    verify {

```

```
    Obj(5).extensionFunc()  
}  
}
```

To mock module-wide extension functions you need to build `mockkStatic(...)` with the module's class name as an argument. For example "pkg.FileKt" for module `File.kt` in the `pkg` package.

```
data class Obj(val value: Int)  
  
// declared in File.kt ("pkg" package)  
fun Obj.extensionFunc() = value + 5  
  
mockkStatic("pkg.FileKt")  
  
every {  
    Obj(5).extensionFunc()  
} returns 11  
  
assertEquals(11, Obj(5).extensionFunc())  
  
verify {  
    Obj(5).extensionFunc()  
}
```

In `jvm` environments you can replace the class name with a function reference:

```
mockkStatic(Obj::extensionFunc)
```

Note that this will mock the whole `pkg.FileKt` class, and not just `extensionFunc`.

This syntax also applies for extension properties:

```
val Obj.squareValue get() = value * value  
  
mockkStatic(Obj::squareValue)
```

If `@JvmName` is used, specify it as a class name.

KHttp.kt:

```
@file:JvmName("KHttp")
```

```
package khttp
// ... KHttp code
```

Testing code:

```
mockkStatic("khttp.KHttp")
```

Sometimes you need to know a little bit more to mock an extension function. For example the extension function `File.endsWith()` has a totally unpredictable `classname`:

```
mockkStatic("kotlin.io.FilesKt__UtilsKt")
every { File("abc").endsWith(any<String>()) } returns true
println(File("abc").endsWith("abc"))
```

This is standard Kotlin behaviour that may be unpredictable. Use `Tools -> Kotlin -> Show Kotlin Bytecode` or check `.class` files in JAR archive to detect such names.

Varargs

From version 1.9.1, more extended vararg handling is possible:

```
interface ClsWithManyMany {
    fun manyMany(vararg x: Any): Int
}

val obj = mockk<ClsWithManyMany>()

every { obj.manyMany(5, 6, *varargAll { it == 7 }) } returns 3

println(obj.manyMany(5, 6, 7)) // 3
println(obj.manyMany(5, 6, 7, 7)) // 3
println(obj.manyMany(5, 6, 7, 7, 7)) // 3

every { obj.manyMany(5, 6, *anyVararg(), 7) } returns 4

println(obj.manyMany(5, 6, 1, 7)) // 4
println(obj.manyMany(5, 6, 2, 3, 7)) // 4
println(obj.manyMany(5, 6, 4, 5, 6, 7)) // 4

every { obj.manyMany(5, 6, *varargAny { nArgs > 5 }, 7) } returns 5

println(obj.manyMany(5, 6, 4, 5, 6, 7)) // 5
```

```
println(obj.manyMany(5, 6, 4, 5, 6, 7, 7)) // 5

every {
    obj.manyMany(5, 6, *varargAny {
        if (position < 3) it == 3 else it == 4
    }, 7)
} returns 6

println(obj.manyMany(5, 6, 3, 4, 7)) // 6
println(obj.manyMany(5, 6, 3, 4, 4, 7)) // 6
```

Private functions mocking / dynamic calls

If you need to mock private functions, you can do it via a dynamic call.

```
class Car {
    fun drive() = accelerate()

    private fun accelerate() = "going faster"
}

val mock = spyk<Car>(recordPrivateCalls = true)

every { mock["accelerate"]() } returns "going not so fast"

assertEquals("going not so fast", mock.drive())

verifySequence {
    mock.drive()
    mock["accelerate"]()
}
```

If you want to verify private calls, you should create a `spyk` with `recordPrivateCalls = true`

Additionally, a more verbose syntax allows you to get and set properties, combined with the same dynamic calls:

```
val mock = spyk(Team(), recordPrivateCalls = true)

every { mock.getProperty "speed" } returns 33
every { mock.setProperty "acceleration" value less(5) } just runs
justRun { mock.invokeNoArgs "privateMethod" }
every { mock.invoke "openDoor" withArguments listOf("left", "rear") } returns "OK"
```

```

verify { mock getProperty "speed" }
verify { mock setProperty "acceleration" value less(5) }
verify { mock invoke "openDoor" withArguments listOf("left", "rear") }

```

Property backing fields

You can access the backing fields via `fieldValue` and use `value` for the value being set.

Note: in the examples below, we use `propertyType` to specify the type of the `fieldValue`. This is needed because it is possible to capture the type automatically for the getter. Use `nullablePropertyType` to specify a nullable type.

Note: This is only for public fields. It is nearly impossible to mock private properties as they don't have getter methods attached. Use Java reflection to make the field accessible or use `@VisibleForTesting` annotation in the source.

```

val mock = spyk(MockCls(), recordPrivateCalls = true)

every { mock.property } answers { fieldValue + 6 }
every { mock.property = any() } propertyType Int::class answers { fieldValue += value
every { mock getProperty "property" } propertyType Int::class answers { fieldValue +
every { mock setProperty "property" value any<Int>() } propertyType Int::class answe
every {
    mock.property = any()
} propertyType Int::class answers {
    fieldValue = value + 1
} andThen {
    fieldValue = value - 1
}

```

Multiple interfaces

Adding additional behaviours via interfaces and stubbing them:

```

val spy = spyk(System.out, moreInterfaces = arrayOf(Runnable::class))

spy.println(555)

every {
    (spy as Runnable).run()
} answers {
    (self as PrintStream).println("Run! Run! Run!")
}

```

```

}

val thread = Thread(spy as Runnable)
thread.start()
thread.join()

```

Mocking Nothing

Nothing special here. If you have a function returning `Nothing`:

```

fun quit(status: Int): Nothing {
    exitProcess(status)
}

```

Then you can for example throw an exception as behaviour:

```

every { quit(1) } throws Exception("this is a test")

```

Clearing vs Unmocking

- clear - deletes the internal state of objects associated with a mock, resulting in an empty object
- unmock - re-assigns transformation of classes back to original state prior to mock

Scoped mocks

A Scoped mock is a mock that automatically unmocks itself after the code block passed as a parameter has been executed. You can use the `mockkObject`, `mockkStatic` and `mockkConstructor` functions.

```

object ObjBeingMocked {
    fun add(a: Int, b: Int) = a + b
}

// ObjBeingMocked will be unmocked after this scope
mockkObject(ObjBeingMocked) {
    assertEquals(3, ObjBeingMocked.add(1, 2))
    every { ObjBeingMocked.add(1, 2) } returns 55
    assertEquals(55, ObjBeingMocked.add(1, 2))
}

```

Suppressing superclass calls

To suppress a method call, especially a `super` call inside an overridden method, you can stub its behavior on a `spyk`. Using `every { ... } just runs` replaces the entire method body, preventing the original code from executing.

This is particularly useful for users coming from frameworks like PowerMockito or for testing classes like Android Activities. For reference, see [PowerMockito suppress documentation](#).

```
// A simple inheritance hierarchy
open class Parent {
    var superCalled = false
    open fun doWork() {
        superCalled = true
    }
}

class Child : Parent() {
    override fun doWork() {
        super.doWork() // We want to suppress this call during testing
    }
}

// In your test:
val child = spyk<Child>()

// Stub the method: prevents the original (super) implementation from being invoked
every { child.doWork() } just runs

child.doWork()

// Verify that the superclass method was not executed
assertFalse(child.superCalled)
```

This approach allows you to isolate the logic within your method for unit testing without executing unwanted parent class behavior.

Matcher extensibility

A very simple way to create new matchers is by attaching a function to `MockKMatcherScope` or `MockKVerificationScope` and using the `match` function:


```
fun MockKMatcherScope.seqEq(seq: Sequence<String>) = match<Sequence<String>> {
    it.toList() == seq.toList()
}
```

It's also possible to create more advanced matchers by implementing the `Matcher` interface.

Custom matchers

Example of a custom matcher that compares list without order:

```
@Test
fun test() {
    class MockCls {
        fun op(a: List<Int>) = a.reversed()
    }

    val mock = mockk<MockCls>()

    every { mock.op(any()) } returns listOf(5, 6, 9)

    println(mock.op(listOf(1, 2, 3)))

    verify { mock.op(matchListWithoutOrder(3, 2, 1)) }
}

data class ListWithoutOrderMatcher<T>(
    val expectedList: List<T>,
    val refEq: Boolean
) : Matcher<List<T>> {
    val map = buildCountsMap(expectedList, refEq)

    override fun match(arg: List<T>?): Boolean {
        if (arg == null) return false
        return buildCountsMap(arg, refEq) == map
    }

    private fun buildCountsMap(list: List<T>, ref: Boolean): Map<Any?, Int> {
        val map = mutableMapOf<Any?, Int>()

        for (item in list) {
            val key = when {
                item == null -> nullKey
                refEq -> InternalPlatform.ref(item)
            }
            map[key] = map.getOrDefault(key, 0) + 1
        }
    }
}
```

```

        else -> item
    }
    map.compute(key, { _, value -> (value ?: 0) + 1 })
}

return map
}

override fun toString() = "matchListWithoutOrder($expectedList)"

@Suppress("UNCHECKED_CAST")
override fun substitute(map: Map<Any, Any>): Matcher<List<T>> {
    return copy(expectedList = expectedList.map { map.getOrDefault(it as Any?, 0) })
}

companion object {
    val nullKey = Any()
}

}

inline fun <reified T : List<E>, E : Any> MockKMatcherScope.matchListWithoutOrder(
    vararg items: E,
    refEq: Boolean = true
): T = match(ListWithoutOrderMatcher(listOf(*items), refEq))

```

Reflection matchers

Example using reflection to mock all methods on a builder-style object

```

val builderFunctions = MyBuilder::class.memberFunctions.filter { it.returnType.class == Void }
val builderMock = mockk<MyBuilder> {
    builderFunctions.forEach { func ->
        every {
            val params = listOf<Any?>(builderMock) + func.parameters.drop(1).map { any(it) }
            func.call(*params.toTypedArray())
        } answers {
            this@mockk
        }
    }
}

```

Configuration File

To adjust parameters globally, you can specify settings in a configuration file.

How to use:

1. Create the configuration file at `src/test/resources/mockk.properties`.
2. Add any of the following configuration options:

```
# MockK Settings
relaxed=true|false
relaxUnitFun=true|false
recordPrivateCalls=true|false
stackTracesOnVerify=true|false
stackTracesAlignment=left|center
failOnSetBackingFieldException=true|false

# Restricted Mocking Configuration (see Restricted Mocking section below)
mockk.restrictedClasses=com.example.MyClass,com.example.AnotherClass
mockk.throwExceptionOnBadMock=true|false
```

Configuration Options

Property	Description	Default Value
<code>relaxed</code>	Enable relaxed mocking globally	<code>false</code>
<code>relaxUnitFun</code>	Enable relaxed mocking for Unit-returning functions	<code>false</code>
<code>recordPrivateCalls</code>	Record private calls for verification	<code>false</code>
<code>stackTracesOnVerify</code>	Show stack traces on verification failures	<code>true</code>
<code>stackTracesAlignment</code>	Align stack traces to <code>left</code> or <code>center</code>	<code>center</code>
<code>failOnSetBackingFieldException</code>	Fail tests if backing field cannot be set	<code>false</code>
<code>mockk.restrictedClasses</code>	Add fully qualified names of classes to restrict from mocking (comma-separated)	N/A

Property	Description	Default Value
<code>mockk.throwExceptionOnBadMock</code>	Throw exception when mocking restricted classes (<code>true</code>), or log warning only (<code>false</code>)	<code>false</code>

Notes:

- `stackTracesAlignment` determines whether to align the stack traces to the center (default), or to the left (more consistent with usual JVM stackTraces).
- If `failOnSetBackingFieldException` is set to `true`, tests fail if a backing field could not be set. Otherwise, only the warning “Failed to set backing field” will be logged. See [here](#) for more details.

Legacy Configuration

For backward compatibility, MockK also supports the legacy configuration file:

```
src/test/resources/io/mockk/settings.properties
```

If both files exist, `mockk.properties` takes precedence. The legacy location is deprecated and will be removed in a future version.

DSL tables

Here are a few tables to help you master the DSL.

Top level functions

Function	Description
<code>mockk<T>(...)</code>	builds a regular mock
<code>spyk<T>()</code>	builds a spy using the default constructor
<code>spyk(obj)</code>	builds a spy by copying from <code>obj</code>
<code>slot</code>	creates a capturing slot

Function	Description
<code>every</code>	starts a stubbing block
<code>coEvery</code>	starts a stubbing block for coroutines
<code>verify</code>	starts a verification block
<code>coVerify</code>	starts a verification block for coroutines
<code>verifyAll</code>	starts a verification block that should include all calls
<code>coVerifyAll</code>	starts a verification block that should include all calls for coroutines
<code>verifyOrder</code>	starts a verification block that checks the order
<code>coVerifyOrder</code>	starts a verification block that checks the order for coroutines
<code>verifySequence</code>	starts a verification block that checks whether all calls were made in a specified sequence
<code>coVerifySequence</code>	starts a verification block that checks whether all calls were made in a specified sequence for coroutines
<code>excludeRecords</code>	exclude some calls from being recorded
<code>confirmVerified</code>	confirms that all recorded calls were verified
<code>checkUnnecessaryStub</code>	confirms that all recorded calls are used at least once
<code>clearMocks</code>	clears specified mocks
<code>registerInstanceFactory</code>	allows you to redefine the way of instantiation for certain object
<code>mockkClass</code>	builds a regular mock by passing the class as parameter
<code>mockkObject</code>	turns an object into an object mock, or clears it if was already transformed
<code>unmockkObject</code>	turns an object mock back into a regular object
<code>mockkStatic</code>	makes a static mock out of a class, or clears it if it was already transformed

Function	Description
<code>unmockkStatic</code>	turns a static mock back into a regular class
<code>clearStaticMockk</code>	clears a static mock
<code>mockkConstructor</code>	makes a constructor mock out of a class, or clears it if it was already transformed
<code>unmockkConstructor</code>	turns a constructor mock back into a regular class
<code>clearConstructorMockk</code>	clears the constructor mock
<code>unmockkAll</code>	unmocks object, static and constructor mocks
<code>clearAllMocks</code>	clears regular, object, static and constructor mocks
<code>clearAllStubsFromMemory</code>	removes all mocks from the internal collection storing them, and as a result frees up memory.

Matchers

By default, simple arguments are matched using `eq()`

Matcher	Description
<code>any<T>()</code>	matches any argument of type <code>T</code> (type-checked)
<code>any(Class)</code>	matches any argument of the given Class (type-checked; useful for reflective mocking)
<code>allAny()</code>	special matcher that uses <code>any()</code> instead of <code>eq()</code> for matchers that are provided as simple arguments
<code>isNull()</code>	checks if the value is null
<code>isNull(inverse=true)</code>	checks if the value is not null
<code>ofType(type)</code>	checks if the value belongs to the type
<code>match { it.startsWith("string") }</code>	matches via the passed predicate

Matcher	Description
<code>coMatch { it.startsWith("string") }</code>	matches via the passed coroutine predicate
<code>matchNullable { it?.startsWith("string") }</code>	matches nullable value via the passed predicate
<code>coMatchNullable { it?.startsWith("string") }</code>	matches nullable value via the passed coroutine predicate
<code>eq(value)</code>	matches if the value is equal to the provided value via the <code>deepEquals</code> function
<code>eq(value, inverse=true)</code>	matches if the value is not equal to the provided value via the <code>deepEquals</code> function
<code>neq(value)</code>	matches if the value is not equal to the provided value via the <code>deepEquals</code> function
<code>refEq(value)</code>	matches if the value is equal to the provided value via reference comparison
<code>refEq(value, inverse=true)</code>	matches if the value is not equal to the provided value via reference comparison
<code>nrefEq(value)</code>	matches if the value is not equal to the provided value via reference comparison
<code>cmpEq(value)</code>	matches if the value is equal to the provided value via the <code>compareTo</code> function
<code>less(value)</code>	matches if the value is less than the provided value via the <code>compareTo</code> function
<code>more(value)</code>	matches if the value is more than the provided value via the <code>compareTo</code> function
<code>less(value, andEquals=true)</code>	matches if the value is less than or equal to the provided value via the <code>compareTo</code> function

Matcher	Description
<code>more(value, andEquals=true)</code>	matches if the value is more than or equal to the provided value via the <code>compareTo</code> function
<code>range(from, to, fromInclusive=true, toInclusive=true)</code>	matches if the value is in range via the <code>compareTo</code> function
<code>and(left, right)</code>	combines two matchers via a logical and
<code>or(left, right)</code>	combines two matchers via a logical or
<code>not(matcher)</code>	negates the matcher
<code>capture(slot)</code>	captures a Non Nullable value to a <code>CapturingSlot</code>
<code>captureNullable(slot)</code>	captures a Nullable value to a <code>CapturingSlot</code>
<code>capture(mutableList)</code>	captures a value to a list
<code>captureNullable(mutableList)</code>	captures a value to a list together with null values
<code>captureLambda()</code>	captures a lambda
<code>captureCoroutine()</code>	captures a coroutine
<code>invoke(...)</code>	calls a matched argument
<code>coInvoke(...)</code>	calls a matched argument for a coroutine
<code>hint(cls)</code>	hints the next return type in case it's gotten erased
<code>anyVararg()</code>	matches any elements in a vararg
<code>varargAny(matcher)</code>	matches if any element matches the matcher
<code>varargAll(matcher)</code>	matches if all elements match the matcher
<code>any...Vararg()</code>	matches any elements in vararg (specific to primitive type)

Matcher	Description
<code>varargAny... (matcher)</code>	matches if any element matches the matcher (specific to the primitive type)
<code>varargAll... (matcher)</code>	matches if all elements match the matcher (specific to the primitive type)

A few special matchers available in verification mode only:

Matcher	Description
<code>withArg { code }</code>	matches any value and allows to execute some code
<code>withNullableArg { code }</code>	matches any nullable value and allows to execute some code
<code>coWithArg { code }</code>	matches any value and allows to execute some coroutine code
<code>coWithNullableArg { code }</code>	matches any nullable value and allows to execute some coroutine code

Validators

Validator	Description
<code>verify { mock.call() }</code>	Do unordered verification that a call was performed
<code>verify(inverse=true) { mock.call() }</code>	Do unordered verification that a call was not performed
<code>verify(atLeast=n) { mock.call() }</code>	Do unordered verification that a call was performed at least <code>n</code> times
<code>verify(atMost=n) { mock.call() }</code>	Do unordered verification that a call was performed at most <code>n</code> times
<code>verify(exactly=n) { mock.call() }</code>	Do unordered verification that a call was performed exactly <code>n</code> times
<code>verifyAll { mock.call1(); mock.call2() }</code>	Do unordered verification that only the specified calls were executed for the mentioned mocks

Validator	Description
<code>verifyOrder { mock.call1(); mock.call2() }</code>	Do verification that the sequence of calls went one after another
<code>verifySequence { mock.call1(); mock.call2() }</code>	Do verification that only the specified sequence of calls were executed for the mentioned mocks
<code>verify { mock wasNot Called }</code>	Do verification that a mock was not called
<code>verify { listOf(mock1, mock2) wasNot Called }</code>	Do verification that a list of mocks were not called

Answers

An Answer can be followed up by one or more additional answers.

Answer	Description
<code>returns value</code>	specify that the matched call returns a specified value
<code>returnsMany list</code>	specify that the matched call returns a value from the list, with subsequent calls returning the next element
<code>returnsArgument(n)</code>	specify that the matched call returns the nth argument of that call
<code>throws ex</code>	specify that the matched call throws an exception
<code>throwsMany list</code>	specify that the matched call throws an exception from the list, with subsequent calls throwing the next exception
<code>answers { code }</code>	specify that the matched call answers with a code block scoped with <code>answer scope</code>
<code>coAnswers { code }</code>	specify that the matched call answers with a coroutine code block with <code>answer scope</code>
<code>answers answerObj</code>	specify that the matched call answers with an Answer object
<code>answers { nothing }</code>	specify that the matched call answers null
<code>just Runs</code>	specify that the matched call is returning Unit (returns null)

Answer	Description
<code>just Awaits</code>	specify that the matched call never returns (available since v1.13.3)
<code>propertyType Class</code>	specify the type of the backing field accessor
<code>nullablePropertyType Class</code>	specify the type of the backing field accessor as a nullable type

Additional answer(s)

A next answer is returned on each consequent call and the last value is persisted. So this is similar to the `returnsMany` semantics.

Additional answer	Description
<code>andThen value</code>	specify that the matched call returns one specified value
<code>andThenMany list</code>	specify that the matched call returns a value from the list, with subsequent calls returning the next element
<code>andThenThrows ex</code>	specify that the matched call throws an exception
<code>andThenThrowsMany ex</code>	specify that the matched call throws an exception from the list, with subsequent calls throwing the next exception
<code>andThen { code }</code>	specify that the matched call answers with a code block scoped with <code>answer scope</code>
<code>coAndThen { code }</code>	specify that the matched call answers with a coroutine code block with <code>answer scope</code>
<code>andThenAnswer answerObj</code>	specify that the matched call answers with an Answer object
<code>andThen { nothing }</code>	specify that the matched call answers null
<code>andThenJust Runs</code>	specify that the matched call is returning Unit (available since v1.12.2)
<code>andThenJust Awaits</code>	specify that the matched call is never returning (available since v1.13.3)

Answer scope

Parameter	Description
<code>call</code>	a call object that consists of an invocation and a matcher
<code>invocation</code>	contains information regarding the actual function invoked
<code>matcher</code>	contains information regarding the matcher used to match the invocation
<code>self</code>	reference to the object invocation made
<code>method</code>	reference to the function invocation made
<code>args</code>	reference to the invocation arguments
<code>nArgs</code>	number of invocation arguments
<code>arg(n)</code>	nth argument
<code>firstArg()</code>	first argument
<code>secondArg()</code>	second argument
<code>thirdArg()</code>	third argument
<code>lastArg()</code>	last argument
<code>captured()</code>	the last element in the list for convenience when capturing to a list
<code>lambda<...>().invoke()</code>	call the captured lambda
<code>coroutine<...>().coInvoke()</code>	call the captured coroutine
<code>nothing</code>	null value for returning <code>nothing</code> as an answer
<code>fieldValue</code>	accessor to the property backing field
<code>fieldValueAny</code>	accessor to the property backing field with <code>Any?</code> type
<code>value</code>	value being set, cast to the same type as the property backing field

Parameter	Description
<code>valueAny</code>	value being set, with <code>Any?</code> type
<code>callOriginal</code>	calls the original function

Vararg scope

Parameter	Description
<code>position</code>	the position of an argument in a vararg array
<code>nArgs</code>	overall count of arguments in a vararg array

Restricted Mocking in MockK

Overview

Restricted Mocking is a feature in MockK designed to **prevent the mocking of classes** that are problematic to mock. These classes often indicate poor test design and can lead to **unreliable** or **misleading test results**.

The primary goal is to:

- **Encourage better testing practices**
- **Promote code maintainability**
- **Avoid mocking classes tied to system operations or critical data structures**

Why Restrict Mocking?

Mocking certain classes can cause several issues:

 Problem	 Impact
False sense of security	Tests may pass even when the implementation is fragile or incorrect.
Tight coupling	Tests become overly dependent on low-level implementation details.

🚩 Problem	⚠️ Impact
Hard-to-maintain tests	Changes in code can break unrelated tests, increasing maintenance overhead.
Code smells	Mocking system-level or value-type classes often signals poor architecture.

Default Restricted Classes

The following classes are **restricted from being mocked by default**:

Class	Description	Includes Subtypes?
<code>java.lang.System</code>	System-related APIs (<code>System.currentTimeMillis()</code> , <code>System.getenv()</code>)	✅ Yes
<code>java.util.Collection</code>	Collections like <code>List</code> , <code>Set</code> , and <code>Queue</code>	✅ Yes
<code>java.util.Map</code>	Key-value data structures like <code>HashMap</code>	✅ Yes
<code>java.io.File</code>	File I/O classes (should be abstracted instead)	✅ Yes
<code>java.nio.file.Path</code>	Path manipulation classes for file systems	✅ Yes

⚠️ **Note:** All subclasses and implementations of these classes are also restricted. For example:

- `ArrayList` and `HashSet` (subtypes of `Collection`)
- `HashMap` (subtype of `Map`)
- Custom classes that extend `File` or implement `Path`

How to Configure Restricted Mocking

You can configure Restricted Mocking behavior using the `mockk.properties` configuration file described in the [Configuration File](#) section.

Add the following properties to your [configuration file](#):

```
# List of restricted classes (fully qualified names, separated by commas)
mockk.restrictedClasses=com.foo.Bar,com.foo.Baz
```

```
# Whether to throw an exception when mocking restricted classes
mockk.throwExceptionOnBadMock=true
```

See the [Configuration File](#) section for detailed information about these and other available options.

⚠ **Note:**

If `mockk.throwExceptionOnBadMock` is not set, it will default to `false`, meaning only warnings will be logged.

To strictly prevent mocking restricted classes, explicitly set:

```
mockk.throwExceptionOnBadMock=true
```

Configure Restricted Mocking with System Properties

You can also configure

```
mockk.throwExceptionOnBadMock=true
```

Example with system properties in your `build.gradle.kts` file:

```
tasks.withType<Test> {
    systemProperty("mockk.throwExceptionOnBadMock", "true")
}
```

You can also do a variable system property and pass the value via the terminal like so:

```
tasks.withType<Test> {
    systemProperty("mockk.throwExceptionOnBadMock", System.getProperty("mockk.throwE
})
```

Then:

```
./gradlew -Dmockk.throwExceptionOnBadMock=true :modules:name:test
```

Configure Restricted Mocking with Gradle Properties

You can add the property to your `gradle.properties` file:

```
systemProp.mockk.throwExceptionOnBadMock=true
```

Then in your terminal:

```
./gradlew -Pmockk.throwExceptionOnBadMock=true :modules:name:test
```

⚠ Note:

The `mockk.throwExceptionOnBadMock` that is set in `gradle.properties` or in your `build.gradle` will override the value that is set in `mockk.properties` file.

Behavior When Mocking Restricted Classes

When `mockk.throwExceptionOnBadMock=false` (Default)

```
@Test
fun `when throwExceptionOnBadMock is false should not throw exception for collection`
    val mockList = mockk<List<String>>()
    every { mockList.size } returns 0

    mockList.size shouldBe 0
}
```

- Result:
 - A warning log is generated, but the test passes.
- Log Example:
 - List should not be mocked! Consider refactoring your test.

When `mockk.throwExceptionOnBadMock=true`

```
@Test
fun `when throwExceptionOnBadMock is true should throw MockKException for collection`
    assertThrows<MockKException> {
        mockk<List<String>>() // Throws MockKException
    }
}
```


- Result:
 - A MockKException is thrown, causing the test to fail.
- Exception Example:
 - MockKException: Mocking java.util.HashMap is not allowed!

Custom Class Restriction Example

You can restrict **custom classes** from being mocked using the `mockk.properties` configuration file. This helps enforce proper testing practices even within your own codebase.

Example 1: Mocking a Restricted Custom Class (Throws Exception)

Add the following to your `mockk.properties` file:

```
package com.foo

class Foo {
    fun doSomething(): String = "print Foo"
}
```

```
package com.bar

class Bar {
    fun doSomething(): String = "print Bar"
}

class Baz : Bar() {
    fun doSomething(): String = "print Baz"
}
```

```
# Restrict custom classes from being mocked
mockk.restrictedClasses=com.foo.Bar,com.foo.Baz

# Throw an exception when attempting to mock restricted classes
mockk.throwExceptionOnBadMock=true
```

```
import io.mockk.mockk
import io.mockk.MockKException
import kotlin.test.Test
import kotlin.test.assertFailsWith
```

```

class RestrictedTest {
    @Test
    fun `should throw exception when mocking restricted class Foo`() {
        assertFailsWith<MockKException> {
            mockk<Foo>() // 🚫 This will throw an exception
        }
    }

    @Test
    fun `should throw exception when mocking restricted class Bar`() {
        assertFailsWith<MockKException> {
            mockk<Bar>() // 🚫 This will throw an exception
        }
    }

    @Test
    fun `should throw exception when mocking restricted class Baz`() {
        assertFailsWith<MockKException> {
            mockk<Baz>() // 🚫 This will throw an exception
        }
    }
}

```

Funding

You can also support this project by becoming a sponsor. Your logo will show up here with a link to your website.

Sponsors



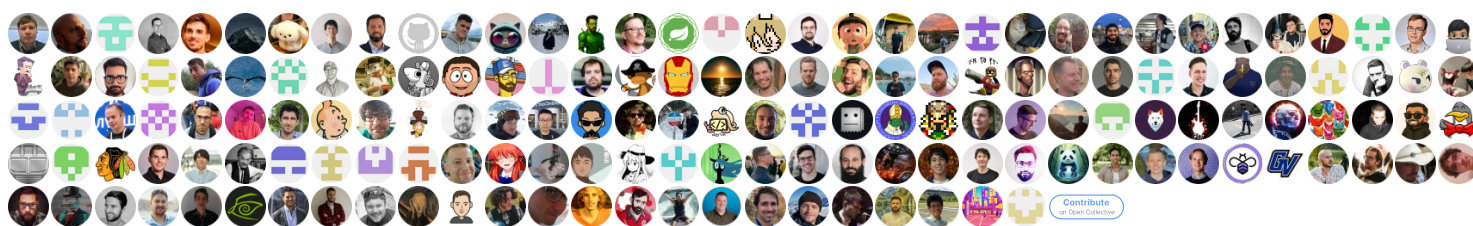
Backers

Thank you to all our backers! 🙏



Contributors

This project exists thanks to all the people who contribute.



Getting Help

To ask questions, please use Stack Overflow or Gitter.

- Chat/Gitter: <https://gitter.im/mockk-io/Lobby>
- Stack Overflow: <http://stackoverflow.com/questions/tagged/mockk>

To report bugs, please use the GitHub project.

- Project Page: <https://github.com/mockk/mockk>
- Reporting Bugs: <https://github.com/mockk/mockk/issues>

Powered by [Java profiler](#), [IntelliJ IDEA](#) and [CLion](#)